# EXPERIMENT 3

## Automated Build and Dependency Management Using Apache Maven

*Eclipse-Based Maven Project with GitHub Actions*

Course: DevOps Laboratory

Duration: 2 Hours

Level: Undergraduate Engineering

Prerequisites: Basic Java programming, Git/GitHub fundamentals, and familiarity with Eclipse IDE.

### 1. Background

Apache Maven is a build automation and dependency-management tool widely used for Java projects. It provides a standard project structure, manages external libraries through the Project Object Model (POM), compiles source code, executes unit tests, and packages applications into deployable artifacts such as JAR files.

In a DevOps workflow, Maven provides the repeatable build step between source-code management and deployment. GitHub stores the source code, Maven performs compilation/testing/packaging, and GitHub Actions automates the Maven build whenever changes are pushed or a pull request is created.

In this experiment, students create a small Java application as an Eclipse Maven project, execute Maven goals from Eclipse, inspect the generated artifact, connect the project to GitHub, and automate the build using GitHub Actions.

### 2. Learning Objectives

- Understand the purpose of Apache Maven in a DevOps workflow.
- Create a Maven project using Eclipse IDE and the Maven project wizard.
- Understand the Maven standard directory structure and pom.xml.
- Manage a project dependency using Maven.
- Compile, test, clean, and package a Java application using Maven.
- Understand the Maven lifecycle and the target directory.
- Integrate the Maven project with GitHub.
- Create a GitHub Actions workflow to automatically build and test the Maven project.

### 3. Learning Outcomes

After completing the experiment, students should be able to:

- Create and configure a Maven project in Eclipse.
- Explain the role of groupId, artifactId, version, packaging, and dependencies in pom.xml.
- Execute Maven build lifecycle phases from Eclipse.
- Write and execute JUnit tests.
- Identify and explain the generated JAR artifact.
- Use Git to publish the Maven project to GitHub.

- Configure GitHub Actions for automated Maven build and test.
- Diagnose a failed build from Maven/GitHub Actions output.

## 4. Software Requirements

- Eclipse IDE for Java Developers with Maven support (m2e).
- JDK 17 or later.
- Apache Maven support through Eclipse/m2e.
- Git.
- A GitHub account.
- Internet connection for downloading Maven dependencies and running GitHub Actions.

## 5. Practical Application: Student Grade Calculator

The experiment uses a small Java application named Student Grade Calculator. The program accepts marks, calculates total and average, and determines whether the student has passed. The application is intentionally small so that students can focus on Maven, testing, Git, and CI automation rather than application complexity.

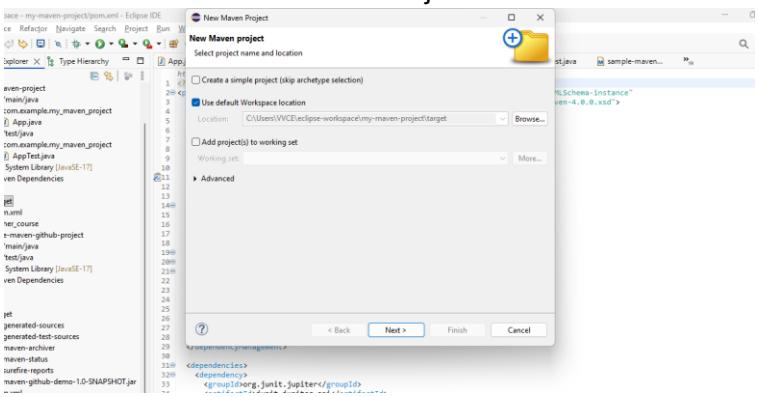
## 6. Part A – Create the Maven Project in Eclipse

### Step 1: Start Eclipse

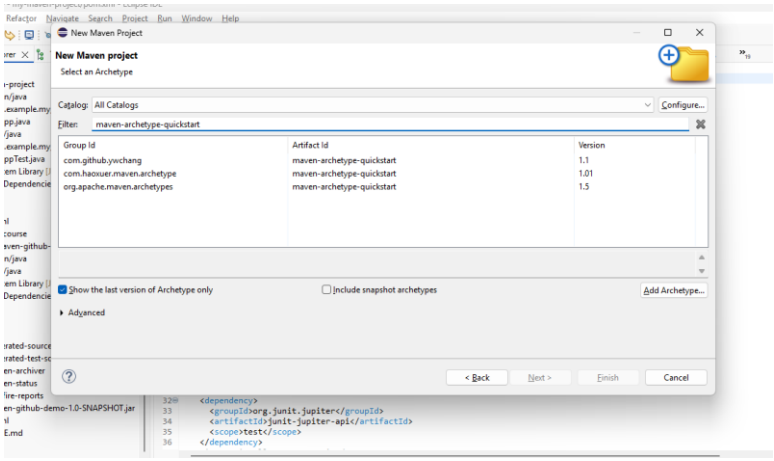
1. Open Eclipse IDE and select an appropriate workspace.
2. Verify that the Java perspective is available.
3. Ensure that Maven support is available in Eclipse.

### Step 2: Create a Maven Project

4. Select File → New → Maven Project.




5. If Maven Project is not directly visible, select File → New → Other → Maven → Maven Project.
6. Click Next.
7. Select Maven Archetype
   In the **Filter** field, type:
   maven-archetype-quickstart
   Select the required quickstart archetype, **version 1.5**, and click **Next**.




## Step 3: Enter Maven Project Details

Use the following project coordinates:

| Field | Value |
|---|---|
| Group Id | com.example |
| Artifact Id | maven-github-demo |
| Version | 1.0-SNAPSHOT |
| Package | com.example |

Click **Finish** , Type Y if Asked

The project is created from the Maven quickstart archetype. Eclipse will initially generate the standard App.java and AppTest.java files. Do not use the generated AppTest.java as the final test because its test framework may be based on the archetype defaults. In the next steps, delete/replace App.java with GradeCalculator.java and delete AppTest.java, then create an explicitly configured JUnit 5 GradeCalculatorTest.java. JUnit 5 is configured directly in pom.xml.

## Step 4: Examine the Maven Project Structure

The project should follow the standard Maven layout:

- src/main/java – application source code.
- src/test/java – unit-test source code.
- pom.xml – Maven Project Object Model containing project configuration and dependencies.
- target – generated build output; it appears after Maven build operations.

```
Maven-github-demo/
├── src/
│   ├── main/
│   │   └── java/
│   │       └── com.example
│   │           └── App.java (replaced it with GradeCalculator.java code)
│   └── test/
```

```
│       └── java/
│           └── com.example
│               └── TestApp.java (replace it with GradeCalculatorTest.java code)
├── pom.xml
└── target/                    (created after build, not present initially)
```

## Step 5: Configure pom.xml

Open pom.xml in Eclipse and replace the archetype-generated test dependency/configuration with an explicit JUnit 5 configuration. Ensure the project uses Java 17 and JUnit 5 for testing.

A suitable configuration is:

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
```

Save pom.xml.

**Note:** Make changes only needed

**Why pom.xml ?**

Example : We want to connect Flink SQL database, it requires JAR file (SQL Connector).

Manual

- Download required JAR
- Add the downloaded JAR to either class path or module path in Eclipse

Pom.xml

- Visit Maven repository website and copy the xml content for that connector and paste in pom.xml. (even transitive dependency will be downloaded)

## Step 6: Replace the Archetype Application with GradeCalculator

Under src/main/java, inside the package com.example : the Maven archetype generated App.java file is present. Replace App.java with this GradeCalculator.java code.

```
package com.example;

public class GradeCalculator {

    public static int calculateTotal(int m1, int m2, int m3) {
        return m1 + m2 + m3;
    }

    public static double calculateAverage(int m1, int m2, int m3) {
        return calculateTotal(m1, m2, m3) / 3.0;
    }

    public static boolean isPass(double average) {
        return average >= 40.0;
    }

    public static void main(String[] args) {
        int m1 = 75, m2 = 68, m3 = 82;

        int total = calculateTotal(m1, m2, m3);
        double average = calculateAverage(m1, m2, m3);

        System.out.println("Total   : " + total);
        System.out.println("Average : " + average);
        System.out.println("Result  : " + (isPass(average) ? "PASS" : "FAIL"));
    }
}
```

## Step 7: Run the Java Program from Eclipse

8. Right-click GradeCalculator.java.
9. Select Run As → Java Application.
10. Observe the Console output.

Expected output will show the total, average, and PASS/FAIL result.

## Step 8: Replace AppTest.java with the JUnit 5 GradeCalculatorTest.java code

Under src/test/java, inside the package com.example, locate the AppTest.java file generated by the Maven quickstart archetype and replace it.

```
package com.example;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class GradeCalculatorTest {
```

```
    @Test
    void testTotal() {
        assertEquals(225,
            GradeCalculator.calculateTotal(75, 68, 82));
    }

    @Test
    void testAverage() {
        assertEquals(75.0,
            GradeCalculator.calculateAverage(75, 68, 82));
    }

    @Test
    void testPass() {
        assertTrue(GradeCalculator.isPass(75.0));
    }

    @Test
    void testFail() {
        assertFalse(GradeCalculator.isPass(35.0));
    }
}
```

### Step 9: Execute JUnit Tests in Eclipse

11. Right-click GradeCalculatorTest.java.
12. Select Run As → JUnit Test.
13. Verify that all tests pass.

Students should inspect the JUnit result window and record the number of tests run, failures, and errors. The test class must run with JUnit 5 (Jupiter), as configured explicitly in pom.xml, rather than relying on the test framework originally supplied by the Maven archetype.

## 7. Part B – Execute Maven Build from Eclipse

### Step 10: Run Maven Clean

14. Right-click the project.
15. Select Run As → Maven build...
16. Enter the goal: clean
17. Click Run.

The clean phase removes previous build output, including the target directory.

### Step 11: Run Maven Test

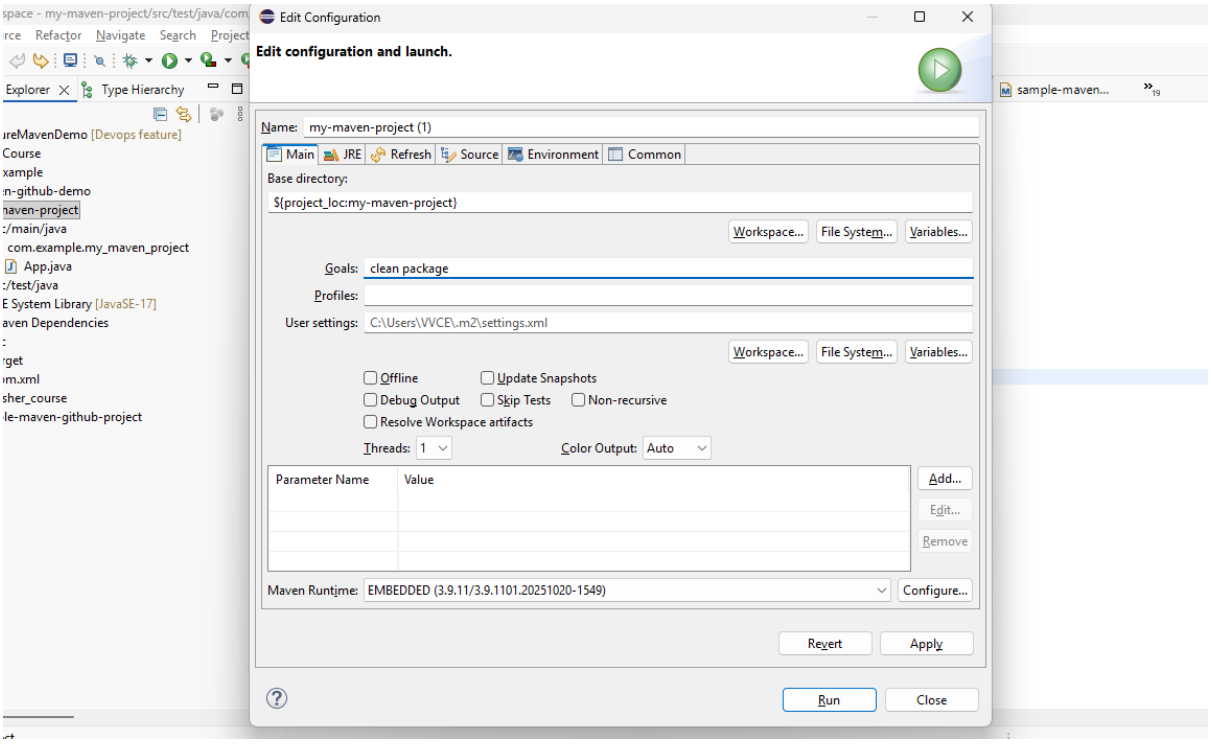
18. Right-click the project.
19. Select Run As → Maven build...
20. Enter the goal: test
21. Click Run.

Maven compiles the testable project and executes the JUnit tests. Examine the Eclipse Console for BUILD SUCCESS.

## Step 12: Run Maven Package

22. Right-click the project and select Run As → Maven build...
23. Enter the goals: clean package
24. Click Run.

The package phase executes the earlier lifecycle phases and creates a JAR file in the target directory.




## Step 13: Inspect the target Directory

Refresh the project in Eclipse and expand target. Students should identify the generated JAR file, compiled classes, test results, and other Maven build output.

```
target/
├── classes/
├── test-classes/
├── surefire-reports/
└── maven-github-demo-1.0-SNAPSHOT.jar
```

The JAR is the build artifact produced by Maven. It can be used as an input to a later deployment stage.

**Note :** Right click on root folder and refresh if the file is not visible

## Step 14: Understand the Maven Lifecycle

The key lifecycle sequence used in this experiment is:

```
clean → compile → test → package
```

- clean – removes previous build output.
- compile – compiles main Java source code.
- test – compiles and runs unit tests.
- package – creates the JAR artifact.

A later lifecycle phase normally invokes the required earlier phases. Therefore, running package performs the necessary compilation and test phases before creating the artifact.

### Step 15: Inspect Dependencies

25. Open the Maven command configuration or Eclipse Maven tooling.
26. Execute the Maven goal: dependency:tree

Observe JUnit and its transitive dependencies. Discuss why dependency management is preferable to manually downloading and maintaining library JAR files.

## 8. Part C – Integrate the Maven Project with GitHub

### Step 16: Initialize the Git Repository

Use the Eclipse Git integration or a terminal opened at the project directory.

(Right click on root folder→ showin→ terminal : to open terminal for entering commands)

```
git init
git add .
git commit -m "Initial Maven project"
```

(Before committing, ensure generated files such as target/ are excluded through .gitignore)

### Step 17: Create a GitHub Repository

27. Sign in to GitHub.
28. Create a new repository named “maven-github-demo”.
29. Keep the repository configuration consistent with the local project.
30. Copy the repository URL.

### Step 18: Connect and Push the Local Repository

```
git branch -M main
git remote add origin <YOUR-GITHUB-REPOSITORY-URL>
git push -u origin main
```

The source code is now stored in the GitHub repository. The Maven build itself is not 'pushed' as a local repository; Git pushes committed repository content to the configured remote repository.

## 9. Part D – Automate Maven Build Using GitHub Actions

### Step 19: Create the Workflow File

In the Eclipse project, under the root folder create the folders .github/workflows and the file maven-build.yml.

```
.github/
└── workflows/
    └── maven.yml
```

**Note:** Folders such as .github and .git will be hidden and not visible in eclipse project explorer. In project explorer goto view menu (3 dots at right)→Filters and Customization and uncheck ***.*resources***

## Step 20: Add the GitHub Actions Workflow in maven.yml

```
name: Maven Build

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Build with Maven
        run: mvn clean package
```

This tells GitHub:

1. Download your source code.
2. Install/configure Java.
3. Configure Maven dependency caching.
4. Run: mvn clean package

Save the workflow file, commit it, and push it to GitHub.

## Step 21: Observe the Automated Build

31. Open the GitHub repository.
32. Select the Actions tab.
33. Open the Maven Build workflow.
34. Select the latest workflow run.
35. Inspect each step: checkout, Java setup, and Maven build.

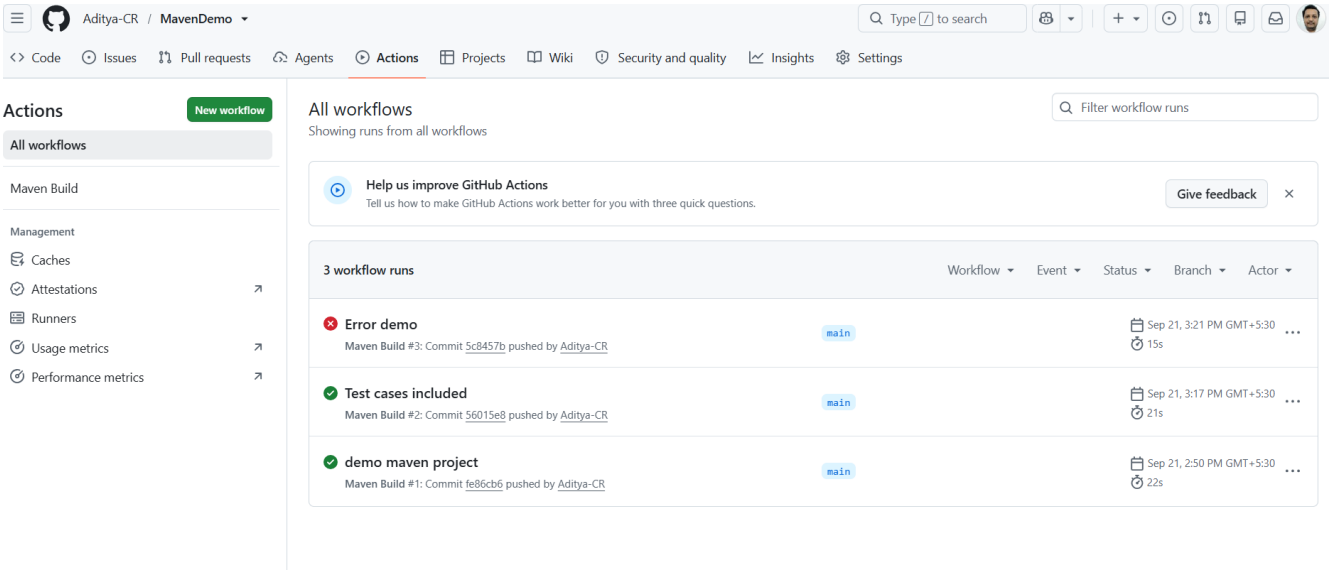
A successful run should show BUILD SUCCESS in the Maven log and a successful workflow status.

## Step 22: Demonstrate a Failed Build

To understand CI feedback, intentionally introduce a controlled error—for example, modify a test so that an expected value is incorrect.

36. Commit and push the change.
37. Open GitHub → Actions.
38. Observe that the Maven build fails at the test/build step.
39. Read the console log to identify the failing test.
40. Correct the test or source code.
41. Commit and push the correction.
42. Verify that the workflow becomes successful.

This demonstrates the DevOps feedback cycle: change → commit → automated build/test → feedback → correction → successful build.




## 10. Final Project Structure

```
student-grade-calculator/
├── .github/
│   └── workflows/
│       └── maven.yml
├── src/
│   ├── main/
│   │   └── java/
│   │       └── com.example
│   │           └── App.java
│   └── test/
│       └── java/
│           └── com.example
│               └── TestApp.java
├── pom.xml
├── .gitignore
└── target/
```

## 11. Key Concepts to Remember

| Component | Purpose |
|---|---|
| Eclipse | IDE used to create, edit, run and manage the Maven project. |
| Maven | Build and dependency-management tool. |
| pom.xml | Defines project metadata, dependencies and build configuration. |
| JUnit | Framework used for unit testing. |
| Git | Version-control system used to track source changes. |
| GitHub | Remote repository hosting platform. |
| GitHub Actions | CI automation platform that executes the Maven build. |
| JAR | Build artifact produced by the package phase. |
| target/ | Directory containing Maven build output. |

## 12. Expected Outputs

- Working Maven project created in Eclipse.
- Successful Java application execution.
- Successful JUnit test execution.
- Successful Maven clean/test/package build.
- Generated JAR file inside target/.
- GitHub repository containing the project source.
- Successful GitHub Actions Maven workflow.
- Evidence of one intentionally failed build and subsequent successful correction.

## 13. Viva Questions

- What is Maven and why is it used in a DevOps workflow?
- What is the purpose of pom.xml?
- What are groupId, artifactId and version?
- What is the standard Maven project structure?
- What happens during the test phase?
- What is the purpose of the package phase?
- Why is target/ generally not committed to Git?
- What is a Maven dependency?
- What is the difference between Git and GitHub?
- What triggers the GitHub Actions workflow in this experiment?
- What is a CI pipeline?
- What is the purpose of actions/checkout?
- Why is JDK setup required in the GitHub Actions runner?

- What happens when a unit test fails in the CI workflow?
- What is a build artifact?
- Why might a JAR be uploaded as a GitHub Actions artifact?

## 14. Teaching Flow for a 2-Hour Lab

| Activity | Approx. Time | Focus |
|---|---|---|
| Introduction and Maven concepts | 10 min | Maven, POM, lifecycle, dependency management |
| Create Maven project in Eclipse | 20 min | Project wizard, coordinates, structure |
| Java program and JUnit tests | 25 min | Application and unit testing |
| Maven clean/test/package | 20 min | Lifecycle and JAR artifact |
| Git/GitHub integration | 15 min | Commit, remote, push |
| GitHub Actions | 20 min | Automated build and test |
| Failure demonstration and discussion | 10 min | CI feedback and troubleshooting |

## 15. Important Conceptual Distinction

Students should clearly distinguish the roles of the tools used in this experiment:

```
Eclipse        → Develops and manages the Java/Maven project
Maven          → Builds, tests and packages the application
Git            → Tracks source-code changes
GitHub         → Hosts the remote Git repository
GitHub Actions→ Automates the Maven build and test
JAR            → Build artifact produced by Maven
```

The experiment therefore demonstrates a complete introductory CI-oriented flow: develop in Eclipse → build/test with Maven → commit and push with Git → automatically build/test through GitHub Actions → produce a JAR artifact.

## 16. Result

The Student Grade Calculator Maven project was successfully created in Eclipse using the Maven quickstart archetype. The archetype-generated App.java and AppTest.java were replaced with GradeCalculator.java and an explicitly configured JUnit 5 GradeCalculatorTest.java. The project was tested using JUnit 5, built and packaged using Maven, pushed to GitHub, and automatically built and tested using GitHub Actions. The generated JAR demonstrates the transition from source code to a deployable build artifact.